

SkinSense AI

Project Log: Training, Quantization, and Raspberry Pi Deployment

A HAM10000 skin-lesion classifier (7 classes) — trained with MobileNetV3 and Quantization-Aware Training on a desktop RTX 3060, exported to ONNX INT8, and deployed to a Raspberry Pi.

Desktop: Ryzen 5 5600X · RTX 3060 12GB · 48GB RAM | Deployment target: Raspberry Pi (Debian 13 “trixie”, aarch64)

1. Project Overview

SkinSense AI classifies dermoscopic skin lesion images into 7 categories using the HAM10000 dataset (10,015 images: akiec, bcc, bkl, df, mel, nv, vasc). The model (MobileNetV3-Large) was trained with Quantization-Aware Training (QAT) so it could be quantized to INT8 for fast, lightweight inference — ultimately for deployment to a Raspberry Pi. The project moved through three broad phases:

- **Training pipeline** — data preprocessing, QAT training, and evaluation, with a long series of bugs found and fixed along the way (environment, training-logic, and logging bugs).
- **ONNX export for Raspberry Pi** — since the desktop's PyTorch build only supports the *onednn* quantization backend (not *qnnpack*, which ARM needs), the QAT-adapted float weights were exported to ONNX and quantized separately by ONNX Runtime.
- **Raspberry Pi deployment** — setting up the Pi's Python environment and running real inference on a test image.

Folder layout: `datasets/` (raw input, untouched) alongside `outputs/` (checkpoints, logs, plots, splits, label encoder, RPi export), configurable via `SKINSENSE_DATA_DIR` / `SKINSENSE_OUTPUT_DIR` environment variables.

2. Setup & Environment Fixes

Before training could run at all, several environment-level issues had to be resolved.

Issue	Fix
<code>requirements.txt</code> had <code>torch torchvision torchaudio --index-url ...</code> all on one line — invalid pip syntax.	Split onto separate lines; added missing <code>colorama</code> dependency; updated from the dropped <code>cu121</code> wheel index to <code>cu126</code> .
<code>pip install ... --index-url</code> caused real resolver failures — PyTorch's wheel index has inconsistent casing for ancillary deps (<code>Jinja2</code> vs <code>jinja2</code>), which pip's strict resolver rejected, causing endless backtracking.	Switched to <code>--extra-index-url</code> instead.
<code>BASE_DIR</code> was hardcoded to a Windows path (<code>F:\Skinsense AI</code>) with mismatched subfolder names (<code>HAM10000_Part1</code> vs the actual <code>HAM10000_images_part_1</code>).	Made configurable via environment variable; fixed subfolder names.

Note: the uploaded project zip contained the training scripts and metadata CSVs but not the real JPGs (only file-listing placeholders) — confirmed the real images already existed locally before proceeding.

3. Training Pipeline Bugs

3.1 Dead configuration values

- **TrainingConfig.BATCH_SIZE was dead code** — the actual `DataLoaders` were built inside `preprocessing_optimized.main()` using its own separate `BATCH_SIZE` constant, so changing the config value did nothing. *Fixed by pushing the value into the preprocessing module before loading data.*
- **config.DROPOUT was dead specifically for MobileNetV3** — the code replaced only the final `Linear` layer, leaving `torchvision`'s stock `Dropout(0.2)` in place regardless of configuration. *Fixed to respect the configured value.*

3.2 QAT / AMP incompatibility and quantized-model crash

- **QAT + AMP conflict** — `USE_QAT=True` and `USE_AMP=True` were both on by default, but they're incompatible in eager-mode quantization (fake-quant ops don't reliably support fp16 autocast). *Now auto-disables AMP whenever QAT is on.*
- **Quantized model crashed on first successful run** with `NotImplementedError: Could not run 'aten::add.out' with... QuantizedCPU`. Root cause: plain mobilenet_v3_large's residual connection uses a raw `+=`, which has no quantized kernel. *Fixed by switching to torchvision's `models.quantization.mobilenet_v3_large / resnet50 (uses FloatFunctional for the residual add)`, plus adding the missing `fuse_model()` call before `prepare_qat`.*

3.3 Catastrophic training collapse

An earlier fix to freeze BatchNorm running stats (`freeze_bn_stats`, standard QAT practice) was scheduled too early — 33% through training — freezing immature statistics while weights were still rapidly adapting. Accuracy collapsed from 81% to 8% within a single epoch and never recovered. Fixed with two changes:

- Moved the BatchNorm freeze much later in training (70% through).
- Added a genuine automatic **collapse-recovery mechanism**: detects a >50% relative accuracy drop and restores model weights from the last good checkpoint (not just toggling flags back), clearing optimizer momentum in place without breaking the LR scheduler's binding to the optimizer. Capped at 3 recoveries per run.

3.4 Root-caused: the quantized accuracy gap

Even after training stabilized, quantized test accuracy consistently lagged float accuracy by 7–10 points, with a clear majority-class bias toward `nv` in the confusion matrix. Multiple training-side timing fixes (BN-freeze / observer-disable spacing) did not close the gap.

Actual root cause (found by reading PyTorch's own source): the `'onednn'` quantization backend's default QAT config leaves the activation observer's `reduce_range` at `False`, unlike `fbgemm/x86` which explicitly use `True`. PyTorch's own docs warn this risks `int8` accumulator overflow/saturation on CPUs without Intel VNNI instructions — which no AMD Ryzen CPU has, including the 5600X.

Fix: explicitly built the `onednn` qconfig with `reduce_range=True`. This fully closed the gap: quantized accuracy went from ~73% to **80.74%**, actually exceeding float test accuracy (80.01%) on the run that used this fix.

3.5 Logging bugs

- Every internal `write()` call got its own timestamp (`print()` calls `write()` separately for text and the trailing newline), shredding log lines apart. *Fixed to timestamp only at true line starts.*
- Log file opened in Windows text mode, silently converting `\n` → `\r\n`. *Fixed with `newline=''`.*
- Only `stdout` was redirected to the log file, never `stderr` — meaning Python crash tracebacks never reached the log. This was the actual reason a real training crash left zero trace in the saved log. *Fixed by redirecting both streams.*
- Console color-coding used substring-anywhere matching (e.g. “contains the word ‘failed’”), which misfired on messages like “Successfully cached 800 images (failed: 0)” — colored red despite zero failures. *Fixed to classify by message prefix instead.*
- A warning/backend-detection block sat at module level, causing it to print 8 times per run — Windows DataLoader workers (spawn method) re-import and re-execute the whole script, once per worker (`NUM_WORKERS=8`). *Fixed by moving all setup into an `initialize_config()` function called only from the entry point.*
- `tqdm` progress bars rendered as plain ASCII # bars instead of solid Unicode blocks — traced to the custom log-mirroring `Tee` class not exposing an `.encoding` attribute, which `tqdm` checks to decide bar style. *Fixed by adding `encoding/isatty()/fileno()` passthrough.*
- Image-caching step (loading up to 2,041 images) had no progress feedback at all. *Added a `tqdm` bar.*

4. Features Added

- **RPi deployment path** (`export_for_rpi.py` + `rpi_inference.py`) — since PyTorch's quantized weights are backend-specific and not portable across quantization engines, the export path sidesteps this by exporting QAT-adapted float weights to ONNX and letting ONNX Runtime (which ships proper ARM/NEON int8 kernels for Raspberry Pi OS) do the quantization instead.
- **Class-weighted loss** — computed from actual post-balancing class counts, since the 'hybrid' balancing strategy only upsamples classes below the median and never downsamples the majority class (nv stayed ~62% of the training set even after “balancing”).
- **Label smoothing (0.1)** and **extra augmentation** (translate/scale jitter, RandomErasing) — general overfitting countermeasures.
- **Progressive/two-stage fine-tuning** (`FREEZE_BACKBONE_EPOCHS=3`) — train only the new classifier head for the first few epochs before unfreezing the pretrained backbone.
- **Human-readable log filenames** (e.g. `training_mobilenet_v3_2026-07-26_09-13AM.log` instead of `log_20260725_205443.txt`).

5. Training Run History

Run	Model	Batch/LR	Val Acc	Float	Quant.	Notes
1	MobileNetV3	64 / 1e-3	83.87%	—	crashed	Quantization crash (fixed after)
2	ResNet50	48 / 1e-3	83.75%	82.07 %	74.67 %	9x longer than MobileNetV3 for no gain; revealed the quant gap
3	MobileNetV3	128 / 1.4e-3	76%*	74%*	72%*	Catastrophic collapse at epoch 11 (BN-freeze timing bug)
4	MobileNetV3	128 / 1.4e-3	81.38%	83.19 %	73.40 %	Collapse fixed; VRAM experiment gave no speed benefit
5	MobileNetV3	64 / 1e-3	82.88%	82.36 %	73.15 %	Reverted batch/LR; widened BN-freeze gap (wrong hypothesis)
6	MobileNetV3	64 / 1e-3	80.13%	80.01 %	80.74 %	onednn <code>reduce_range</code> fix — quantization gap closed

*Run 3's numbers reflect the collapsed/recovered state, not a clean result. Run 6 (highlighted) is the final configuration used for deployment.

Final configuration

- Model: MobileNetV3-Large (torchvision) · Batch size: 64 · LR: 1e-3 · Weight decay: 1e-4
- Dropout: 0.3 · Label smoothing: 0.1 · Class-weighted CrossEntropyLoss
- `FREEZE_BACKBONE_EPOCHS = 3` (progressive unfreezing)
- QAT enabled, backend onednn, with the `reduce_range=True` fix
- BatchNorm freeze at 70% through training, observer disable at ~90%
- Automatic collapse-recovery safety net (max 3 recoveries/run)
- 30 epochs, early stopping patience 10

Quantized accuracy target (>80%) was met: **80.74%**, matching/slightly exceeding float accuracy. This is in line with published results for plain (non-ensembled) lightweight CNNs on HAM10000 under a correct lesion-wise train/test

split — the commonly-quoted 90%+ numbers for this dataset typically come from non-lesion-wise splits that leak same-lesion images between train/test.

6. ONNX Export for Raspberry Pi — The Debugging Journey

This turned out to be the most involved part of the whole project. Two separate, unrelated export failures had to be diagnosed and fixed in sequence before the ONNX file would produce successfully. Environment: PyTorch 2.13.0+cu126.

6.1 Failure #1 — GuardOnDataDependentSymNode

The first attempt at `torch.onnx.export(...)` crashed with:

```
torch.fx.experimental.symbolic_shapes.GuardOnDataDependentSymNode:
Could not guard on data-dependent expression Eq(u0, 1)
```

Diagnosis: the QAT-prepared model still has `FakeQuantize` modules attached (as forward hooks and as direct submodules like `weight_fake_quant`) even after calling `disable_fake_quant` / `disable_observer` — those calls only flip internal boolean flags, they don't remove the modules. `FakeQuantize.forward()` unconditionally starts with a plain Python `if self.observer_enabled[0] == 1:` on a tensor value. The old TorchScript-based ONNX exporter just evaluates this as a concrete bool at trace time; the newer `torch.export/dynamo`-based exporter (the default in recent PyTorch) instead tries to resolve it as a symbolic guard and fails.

Fix attempt 1: pass `dynamo=False` to `torch.onnx.export(...)` to force the legacy tracer, which evaluates the flag as a real Python bool instead of a symbolic one. This worked — the legacy tracer successfully traced the *entire* model graph, all the way to the ONNX conversion step.

6.2 Failure #2 — UnsupportedOperatorError

With the tracer issue solved, a second, different error appeared:

```
torch.onnx.errors.UnsupportedOperatorError: Exporting the operator
'aten::fused_moving_avg_obs_fake_quant' to ONNX opset version 17
is not supported
```

Diagnosis: not an opset-version problem (bumping to 18 wouldn't help) — the ONNX exporter simply has no symbolic mapping for this op at any opset. The disabled `FakeQuantize` modules were still present in the graph and still traced as this exact op, repeated on every single quantized layer.

Fix: since fake-quant and observer were already disabled, `FakeQuantize(x)` is mathematically just `x` — so every `FakeQuantizeBase` instance in the model was recursively swapped for `nn.Identity()`. This removes the untraceable op from the graph with zero numerical difference (the conv weights exported are already the BN-fused `scaled_weight` computed inside `ConvBn2d/ConvBnReLU2d`; the `FakeQuantize` call after that was always a passthrough here).

```
def strip_fake_quant(module):
    for name, child in module.named_children():
        if isinstance(child, torch.ao.quantization.FakeQuantizeBase):
            setattr(module, name, torch.nn.Identity())
        else:
            strip_fake_quant(child)
    strip_fake_quant(model)
```

6.3 Result

With both fixes in place, the export completed cleanly on the first run:

File	Purpose	Size
------	---------	------

mobilenet_v3_float.onnx	Full-precision reference	16.3 MB
mobilenet_v3_int8_rpi.onnx	Deployed to the Pi	4.5 MB (~3.6x smaller)
label_encoder.pkl	Class-name mapping, copied alongside	286 bytes

Quantization used here is **dynamic** (weights only; activations still compute in float) via ONNX Runtime's `quantize_dynamic`. Static quantization (calibrated on real validation images, quantizing activations too) was noted as a possible future speed improvement but not pursued.

7. Raspberry Pi Deployment

Target device: Debian GNU/Linux 13 “trixie” (aarch64), Python 3.13.

7.1 Steps taken

- Copied the `rpi_export` folder to the Pi, plus `rpi_inference.py` (which lives in the project root, not the export folder, and had to be added separately).
- Created a virtual environment: `python3 -m venv skinsense_env && source skinsense_env/bin/activate`.
- Installed dependencies: `pip install onnxruntime pillow numpy`. A prebuilt aarch64 wheel for `onnxruntime` 1.28.0 was available directly from PyPI/piwheels — no manual ARM build was needed.

7.2 sklearn dependency gap

First run failed immediately on unpickling the label encoder:

```
ModuleNotFoundError: No module named 'sklearn'
```

`label_encoder.pkl` is a pickled scikit-learn `LabelEncoder` object, so scikit-learn has to be installed on the Pi purely to unpickle it (even though the Pi never runs any sklearn code itself). Fixed with `pip install scikit-learn`. This pulled in `scipy`, `joblib`, `threadpoolctl`, and `narwhals` as transitive dependencies.

The Pi installed scikit-learn 1.9.0 while the desktop trained with 1.7.2, which produced a non-fatal `InconsistentVersionWarning` on unpickling. Inference still worked correctly, but pinning `scikit-learn==1.7.2` on the Pi would remove the warning and eliminate a theoretical (if here harmless) forward-compatibility risk.

7.3 First successful inference

GPU-detection warnings from ONNX Runtime probing `/sys/class/drm/card0/card1` appeared but are cosmetic — the Pi has no usable discrete GPU for this path, and the script explicitly requests `CPUExecutionProvider` anyway.

Result on test image `ISIC_0029307.jpg`:

Class	Probability
nv (predicted)	51.7%
vasc	11.2%
akiec	10.9%
df	10.8%
bcc	9.7%
bkl	3.7%

mel	2.1%
-----	------

Two honest caveats noted about this result: (1) `nv` was the majority class even after balancing (~62% of training data pre-balancing), so some baseline pull toward it is expected; (2) 51.7% is a fairly low-confidence call, consistent with the model's 80.74% overall accuracy — this particular image likely sits near a decision boundary rather than indicating a pipeline problem.

8. Current Status & Open Items

Working end-to-end: training → QAT → ONNX export → INT8 quantization → Raspberry Pi inference, with a verified prediction on a real test image.

Open items, not yet acted on:

- Side-by-side comparison of the float vs. INT8 ONNX model on the same image(s), to confirm quantization hasn't skewed any specific prediction.
- Optional: static quantization (calibrated on real validation images) for a bigger inference speed win over the current dynamic quantization.
- Optional: pin `scikit-learn==1.7.2` on the Pi to match the training environment exactly and silence the unpickling version warning.
- Optional: run ONNX Runtime's suggested pre-processing (`quant_pre_process`) on the float model before quantizing — not required, but may improve quantization accuracy slightly.

End of report.